

7 Vulnerabilities of "Vibe-Coded" Apps

A quick security guide for creators, developers, and AI builders by Code School by Rishabh.

SUBSCRIBE ON YOUTUBE

The Security Risk Checklist

Apps built rapidly using LLMs and AI Agents ("vibe-coded" apps) often focus solely on functionality while ignoring non-functional security primitives. This field guide lays out the top 7 vulnerabilities found in modern AI-generated logic, their severity, and immediate preventions.

CRITICAL SEVERITY

1. Server-Side Template Injection (SSTI)

Simplified: Agar hacker ne login field mein code daala aur tumhara server chalne laga, toh pura laptop unka ho jayega.

The Exploit:

AI templates often display friendly login errors by concatenating raw user inputs directly into string interpolation fields of rendering engines (e.g., EJS, Pug, Jinja) instead of passing variables safely as data parameters.

Prevention:

- Never use raw string interpolation or evaluation logic (`eval()` or custom template rendering) on incoming request payloads.
- Utilize built-in contextual auto-escaping or pass raw values exclusively via secure, sandboxed parameters.

CRITICAL SEVERITY

2. Regular Expression Denial of Service (ReDoS)

***Simplified:** AI ka ganda validation code ek lamba email string jhel nahi payega, aur tumhari poori app freeze ho jayegi.*

The Exploit:

Unoptimized regex generated by AI assistants can collapse under catastrophic backtracking when evaluated against intentionally malformed strings, trapping the single-threaded Node.js event loop.

Prevention:

- Use standard, battle-tested validation suites (like validator.js) instead of prompting AI to construct custom lookahead regular expressions.
- Implement aggressive response timeouts on application processes and test regex using static analysis analyzers.

CRITICAL SEVERITY

3. Long Password DoS

***Simplified:** Agar kisi ne 10,000 alphabets ka password bhej diya, toh tumhara server usko process karte-karte thak ke crash ho jayega.*

The Exploit:

Failing to cap input sizes before processing them under computationally heavy cryptographic algorithms (such as bcrypt or scrypt) causes complete host CPU exhaustion from a single payload.

Prevention:

- Enforce a hard max length (e.g., 128 characters) at your routing validation layer **before** hitting the database or hashing module.
- Drop oversized request content using an upstream middleware or proxy configuration.

CRITICAL SEVERITY

4. AWS S3 Client-Side Secret Leakage

Simplified: Apni AWS keys galti se frontend code mein chhod di? Pura cloud database leak ho chuka hai.

The Exploit:

Using framework environment prefixes improperly (like appending NEXT_PUBLIC_ or VITE_ onto private environment parameters) exposes AWS root credentials directly in the compiled public JS bundle.

Prevention:

- Always restrict credential parameters strictly to runtime environment files (.env) and build-time checks.
- Utilize AWS IAM Roles and ephemeral credentials rather than static root keys in codebases.

HIGH SEVERITY

5. NoSQL Injection

Simplified: Hacker login screen par bina password ke sirf ek bracket daalkar seedha admin account mein ghus jayega.

The Exploit:

Accepting raw un-sanitized JSON request objects directly into MongoDB query logic allows attackers to bypass strings and pass relational queries like { "\$gt": "" } to sign into accounts automatically.

Prevention:

- Enforce strict request schemas with schema validators (such as Zod or Joi) to ensure fields are plain strings.
- Sanitize data fields using dedicated mongo-sanitize plugins.

HIGH SEVERITY

6. Clipboard Copy Attack (Pastejacking)

Simplified: Tumhari website se code copy karte hi hacker piche se apna virus code chipka dega, aur terminal kholte hi chal jayega.

The Exploit:

Malicious web code manipulation or background endpoint exploits modify the operating system clipboard to inject terminal-executable strings secretly behind innocent code blocks.

Prevention:

- Ensure terminal setups use Bracketed Paste Mode to inspect commands safely before execution.
- Build text-copy buttons using secure, programmatic mechanisms (`navigator.clipboard.writeText`) instead of un-sanitized selections.

LOW SEVERITY

7. Login Replay Attack

Simplified: Bina security encryption ke hacker tumhara network data raste mein pakad ke baar-baar bhejega aur bina password ke login kar lega.

The Exploit:

Interception of plain transmission cookies or data blocks in transit allowing malicious third parties to pass un-expired authentications directly into backends without cracking secrets.

Prevention:

- Enforce continuous TLS/HTTPS layers to encrypt transit footprints natively.
- Integrate server nonces or time-based one-time tokens (TOTPs) to automatically expire individual transmission items.



Creator Bonus: The Single-Thread Guardrail

Because your **Node.js** backend runs on a single event loop thread, an unhandled error or blocking operation inside any of these pre-authentication fields crashes your app globally for all users simultaneously.

Pro-Tip: Always run your production applications under a cluster manager like **PM2** or inside auto-scaling AWS ECS tasks. If an un-vetted attack slips past your login validation and triggers a crash, the process manager instantly forks a fresh worker thread, preserving uptime while you fix the underlying code.

VISIT YOUTUBE CHANNEL FOR DEEP DIVES

CODE SCHOOL BY
RISHABH